Uniform PA calibration for constructor coefficients at least four

Supplement to the new 194-state PA-consistency machines

13 November 2026, Pacific time

1 Uniform calibration for formula radices $k \geq 4$

The formula multiplier can be changed from four to five or six without changing the consistency strength of the numerical calculus. This is not justified by replacing the numeral four in the old proof: the old calculus identifies a formula's code with a fresh variable label, and its propositional axiom has a numerical side condition. Both features must be addressed explicitly. The construction below does so for every fixed natural number $k \geq 4$. The literal machines in the accompanying packet instantiate it at $k = 5$ and $k = 6$.

1.1 The numerical calculus at radix k

Let

$$p(a, b) = \frac{(a + b)(a + b + 1)}{2} + b$$

be Cantor pairing, with its usual primitive-recursive inverse. Define

$$\begin{aligned} I_k(A, B) &= kp(A, B), & M_k(a, b) &= kp(a, b) + 1, \\ U_k(a, B) &= kp(B, a) + 2, & E_k(a, b) &= kp(a, b) + 3. \end{aligned}$$

The reversal in U_k is deliberate. Code zero denotes truth; code one, which equals $M_k(0, 0)$, is the distinguished false formula. Put

$$N_k(B) = I_k(B, 1), \qquad A \wedge_k B = N_k(I_k(A, N_k(B))), \qquad \exists_k x_a B = N_k(U_k(a, N_k(B))).$$

In formula displays, implication, negation, conjunction and existential quantification refer to these numerical constructors.

The six retained heads are

$$\begin{aligned} h_1(D, H) &= (\exists_k x_D\, U_k(D, H)) \to H, \\ h_2(D, A, H) &= U_k(D, U_k(A, H)) \to U_k(A, U_k(D, H)), \\ h_3(D, A, H) &= M_k(H, D) \to (E_k(H, A) \to M_k(A, D)), \\ h_4(D, A, H) &= E_k(H, D) \to (M_k(A, H) \to M_k(A, D)), \\ h_5(D, H) &= \exists_k x_D E_k(H, D), \\ h_6(D, A, H) &= U_k(D, A) \to (U_k(D, A \to H) \to U_k(D, H)). \end{aligned}$$

The propositional head is

$$\rho_{8,k}(D, A, H, u) = ((H \to u) \to D) \to ((D \to H) \to (A \to H)), \qquad u \leq D.$$

Define

$$\begin{aligned}
\Gamma_k(B) &= I_k(B, U_k(B, B)),\\
V_k(a,b,c) &= M_k(a,b) \to U_k(c, M_k(a,b)),\\
P_k(a,b,c) &= c \to M_k(c,b),\\
S_k(a,b,c) &= U_k(b, (M_k(a,b) \to P_k(a,b,c)) \wedge_k (M_k(c,b) \to c)),\\
Q_k(a,b,c) &= U_k(c, S_k(a,b,c) \to N_k(P_k(a,b,c))),\\
G_k(a,b,c) &= N_k(V_k(a,b,c) \to Q_k(a,b,c)).
\end{aligned}$$

Let $\mathcal{C}_k$ have these six heads, the dominated propositional heads, every $\Gamma_k(B)$, and $G_k(a,b,c)$ when $c \neq a, b$. Its rules are modus ponens and generalization $B \mapsto U_k(i, B)$ on derivations with no undischarged hypotheses. The formulas themselves may have free variable labels. Failed guarded emissions produce zero, which is already derivable since $\rho_{8,k}(0,0,0,0) = 0$.

1.2 Hereditary recoding of formulas and variable labels

Cantor pairing satisfies $a, b \leq p(a,b)$. Consequently, if $n = kp(a,b) + r > 0$, where $k \geq 4$ and $0 \leq r < k$, then $a, b < n$. This gives the well-founded recursions used below.

Define $F_k(0) = 0$. For $n > 0$, write uniquely $n = 4p(a,b) + r$ with $0 \leq r < 4$, and set

$$F_k(n) = kp(F_k(a), F_k(b)) + r. \tag{R1}$$

This is course-of-values primitive recursion. For example, a primitive recursion on n stores the already computed sequence $F_k(0), \ldots, F_k(n-1)$ and obtains the two required previous values by primitive-recursive lookup. Thus F_k is primitive recursive, jointly in k and n on the stated parameter domain. PA proves its defining identities by induction. It fixes $0, 1, 2, 3$, and induction on n also proves $F_k(n) > 0$ for $n > 0$.

For injectivity, define a second primitive-recursive function J_k. Set $J_k(0) = 0$. For $m > 0$, decode $m = kp(a,b) + r$, $0 \leq r < k$, and put

$$J_k(m) = \begin{cases} 4p(J_k(a), J_k(b)) + r, & r < 4,\\ 0, & 4 \leq r < k. \end{cases} \tag{R2}$$

Whenever the recursive clause is used its arguments are smaller than m. Induction on n, using positivity for $n > 0$ and the uniqueness of quotient, remainder and Cantor decoding, proves

$$J_k(F_k(n)) = n. \tag{R3}$$

Therefore F_k is injective. In particular, all numerical equality and inequality guards among source labels are preserved. The function J_k is used only to establish injectivity; it is not asserted to translate arbitrary $\mathcal{C}_k$ proofs back to radix four.

Directly from (R1), including the exceptional case $I_4(0,0) = 0$, we get

$$\begin{aligned}
F_k(I_4(A,B)) &= I_k(F_k(A), F_k(B)),\\
F_k(M_4(a,b)) &= M_k(F_k(a), F_k(b)),\\
F_k(U_4(a,B)) &= U_k(F_k(a), F_k(B)),\\
F_k(E_4(a,b)) &= E_k(F_k(a), F_k(b)).
\end{aligned} \tag{R4}$$

Thus the same map must be applied to a formula's numerical code and to every numerical variable label occurring in it. Injectivity ensures that binding, repeated labels and shadowing are respected: a binder with old label a binds exactly the occurrences that receive new label $F_k(a)$. In particular,

$$F_k(\Gamma_4(B)) = I_k(F_k(B), U_k(F_k(B), F_k(B))) = \Gamma_k(F_k(B)). \tag{R5}$$

This is the reason for using hereditary recoding. Recoding formula constructors while holding the variable labels fixed would not preserve the diagonal axiom. Equation (R4) also preserves each retained head, each of V, P, S, Q, G, modus ponens and generalization; injectivity preserves the guard on G.

1.3 Removing the numerical-order obstruction

The function F_k need not preserve order. For instance,

$$40 = 4p(4,0) < 4p(3,1) = 44, \qquad F_5(40) = 75 > 55 = F_5(44).$$

It would therefore be incorrect to translate a restricted $\rho_{8,4}$ axiom into one restricted $\rho_{8,k}$ axiom merely by recoding its four parameters.

There is, however, a fixed finite derivation of the unrestricted schema from the restricted schema. The archived source `M6/Logic/RestrictedImplication.lean` proves a theorem `rho8All` for arbitrary objects A, an implication operation, a relation $\preceq$, and proof constructors with the following properties:

1. modus ponens is available;
2. the row $((c \to u) \to a) \to ((a \to c) \to (b \to c))$ is available if $u \preceq a$;
3. $a \preceq a$;
4. $u \preceq a$ implies $u \preceq (a \to b)$;
5. $u \preceq b$ implies $u \preceq (a \to b)$.

Its conclusion is that the displayed row has a derivation for *all* a, b, c, u, with no order hypothesis. Its intermediate proofs derive weakening, Peirce's law, composition, exchange and the usual implication distribution law. Every restricted leaf's side condition is built solely by the last three listed operations. The source is a finite proof template; it contains no search, semantic completeness assumption or numerical-code substitution rule.

For $\mathcal{C}_k$, instantiate $\preceq$ by ordinary $\leq$ and implication by I_k. The needed bounds follow from

$$a, b \leq p(a,b) \leq kp(a,b) = I_k(a,b).$$

Transitivity gives the last two properties. This instantiation works for every $k \geq 4$ (indeed, the bound argument alone needs only $k \geq 1$). The fixed finite template is a primitive-recursive proof constructor whose correctness PA verifies.

Translate a finite $\mathcal{C}_4$ derivation by (R4) at all ordinary proof constructors. For each dominated axiom leaf, replace the leaf by this unrestricted template at the recoded parameters. The template's own restricted leaves all have valid numerical side conditions at radix k. Induction on the original derivation gives a primitive-recursive map L_k with

$$\mathrm{PA} \vdash \mathrm{Prf}_{\mathcal{C}_4}(d, A) \;\longrightarrow\; \mathrm{Prf}_{\mathcal{C}_k}(L_k(d), F_k(A)). \tag{R6}$$

In particular, $F_k(1) = 1$ transfers every radix-four refutation to a radix-k refutation. No numerical monotonicity of F_k is used.

1.4 PA soundness, including the unused residues

For $k > 4$ there are natural numbers whose residue is not one of the four constructor tags. They must receive an interpretation, since arbitrary numerical command parameters can place them inside the emitted formulas. Define a total syntactic translation T_k into arithmetic formulas as follows. Translate zero to the closed truth $0 = 0$. For $n > 0$, decode $n = kp(a, b) + r$:

r	$T_k(n)$
0	$T_k(a) \to T_k(b)$
1	$\mathrm{Bit}(x_a, x_b)$
2	$\forall x_b\, T_k(a)$
3	$x_a = x_b$
$4, \ldots, k-1$	$0 = 0$.

(R7)

The auxiliary bound variables inside the bit predicate are renamed above the finitely many source labels, or represented by de Bruijn indices. Thus the labels a, b in (R7) refer to the source variables, with ordinary binding and shadowing semantics. The recursion is primitive recursive because every recursive formula argument is strictly smaller than n.

Every source label occurring in $T_k(B)$ is strictly below B. This follows by induction from the component bound; zero and the unused residues translate to closed truth. In particular x_B is not free in $T_k(B)$, so $T_k(\Gamma_k(B))$ has a uniform PA proof by vacuity. The implication constructor has a uniform PA-provable equivalence

$$T_k(I_k(A, B)) \;\leftrightarrow\; (T_k(A) \to T_k(B)). \tag{R8}$$

It is a syntactic identity except at $A = B = 0$, where the proof is the fixed tautology $(0 = 0) \leftrightarrow ((0 = 0) \to (0 = 0))$. This handles the numerical alias $I_k(0, 0) = 0$ explicitly.

Use the same bit interpretation as in the archived PA calibration. PA proves

$$\mathrm{Bit}(s, z) \to z < s, \qquad \neg\,\mathrm{Bit}(z, z).$$

For each arithmetic formula $\theta(z, \vec{u})$ there is a primitive-recursive supplier of a PA proof of finite comprehension:

$$\forall n\, \exists s < 2^n\, \forall z < n\, (\mathrm{Bit}(s, z) \leftrightarrow \theta(z, \vec{u})). \tag{R9}$$

It uses one PA induction instance on n. From a witness below 2^n, the next witness is either s or $s + 2^n$, according to excluded middle on $\theta(n, \vec{u})$. The proof supplier forms that formula and induction instance syntactically; it never decides θ and assumes no satisfaction or reflection principle.

For clarity, the guarded gate uses precisely the same finite comprehension argument at every radix. Set $\phi = T_k(c)$ and assume $c \neq a, b$. If the labels a, b are distinct, use bound $x_a + x_b + 1$ and matrix

$$(\mathrm{Bit}(x_a, z) \vee z = x_b) \wedge \phi[z/x_b]. \tag{R10}$$

If $a = b$, use bound $x_b + 1$ and matrix

$$(z = x_b) \wedge \phi[z/x_b]. \tag{R11}$$

In the second case the first membership antecedent under the quantifier for label b is $\mathrm{Bit}(z, z)$ and is false. In both cases the finite set witness makes the translation of $S_k(a, b, c)$ true and satisfies the distinguished-point property $P_k(a, b, c)$. The bound on set bits converts the bounded assertions from (R9) into the required unbounded implications. Capture-avoiding substitution preserves the original free values when choosing the witness, and freshness of c in $T_k(c)$ means the choice does not alter

ϕ. We obtain a PA proof of $\neg T_k(Q_k(a,b,c))$. The guard also makes V_k a valid membership-vacuity instance, so classical logic gives $T_k(G_k(a,b,c))$. When the guard fails the emission is zero, with its fixed truth proof.

The other six heads have the same first-order proof suppliers as at radix four. The propositional head is a classical tautology even without its numerical restriction. Modus ponens uses (R8), and generalization uses the ordinary PA rule. These arguments apply to *all* numerical parameter values, including unused-residue formulas, since (R7) interprets those as truth. Finally,

$$T_k(1) = \mathrm{Bit}(x_0, x_0)$$

has a fixed PA refutation, also uniformly under binders for label zero. Hence the source negation and existential abbreviations have their intended classical equivalences.

Recursing on the finite derivation therefore supplies a primitive-recursive refutation translator R_k such that

$$\mathrm{PA} \vdash \mathrm{Prf}_{\mathcal{C}_k}(d,1) \;\longrightarrow\; \mathrm{Prf}_{\mathrm{PA}}(R_k(d), \ulcorner 0 = 1 \urcorner). \tag{R12}$$

This is a syntactic proof-checking theorem, so its universal quantification includes the proof codes in nonstandard models of PA. Combining (R6), (R12), and the archived PA calibration of $\mathcal{C}_4$ gives

$$\mathrm{PA} \vdash (\exists d\ \mathrm{Prf}_{\mathcal{C}_k}(d,1) \;\leftrightarrow\; \exists p\ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner)). \tag{R13}$$

1.5 The ten-entry compiler works unchanged in form

The numerical stack and command-word scheduler continue to use ordinary Cantor pairing, without a multiplier. Only formula construction uses I_k, M_k, U_k, E_k. Write $[a]w = p(a,w)$. Shift tests $t + h = 1$ and otherwise sends (t,w,d,h) to $(h, \mathrm{tl}(w), d, \mathrm{hd}(w))$. For $v = d + x$, modus ponens compares $I_k(t,v)$ with h; on success it returns $(v, [0]w, v, 0)$ and on failure $(0,w,v,0)$.

The unchanged free-term emitter identity says that $\mathsf{AX}(c,x,y)$, with $D = d + x, H = h + y, A = c$, pushes in order

$$t,\ U_k(d,h),\ \rho_{8,k}(D,A,H,d),\ h_1, h_2, h_3, h_4, h_5, h_6, \Gamma_k(A),$$

leaves the guarded $G_k(H,D,A)$ on top, and clears the caches. Let $q_1, \ldots, q_{10}$ denote these stack entries in reverse order. The gate and $q_1, \ldots, q_8$ have residue zero modulo k; $q_9 = U_k(d,h)$ has residue two and is at least two. These facts prove the same safe-shift identities as in the archived compiler:

$$\mathsf{AX}(c,x,y)\mathsf{S}^9(t,w,d,h) = (\rho_{8,k}(D,c,H,d), [t]w, 0, U_k(d,h)),$$
$$\mathsf{AX}(c,x,y)\mathsf{S}^{10}(t,w,d,h) = (U_k(d,h), w, 0, t).$$

Every tested sum is different from one. No primality, parity, or special identity of the number four is involved.

Consequently the same words give

$$\begin{aligned} \mathrm{pop} &= \mathsf{AX}(0,0,0)\mathsf{S}^{11}, \\ \mathrm{push} &= \mathsf{AX}(0,0,0)\mathsf{S}^{9}\mathsf{MP}(0), \\ \mathrm{discard} &= \mathsf{MP}(1)\mathsf{AX}(1,0,0)\mathsf{S}^{10}\mathsf{MP}(0). \end{aligned}$$

The MP comparison in push fails because implication and universal codes have different residues. The two MP comparisons in discard fail because $I_k(t,1) > 0$ and $I_k(U_k(1,0),0) > 0$. For $i > 0$, $\mathsf{MP}(i)$ at $(0,w,0,0)$ sets $d = i$ because $I_k(0,i) > 0$. These are the only numerical facts used by the offset, selector, generalization and MP-suffix constructions.

For an actual restricted leaf with $u \le D$, select its head by

$$\text{discard offset}(u)\,\mathsf{AX}(A, D-u, H)\mathsf{S}^9\text{pop}.$$

The subtraction here is performed on the *target* numerical parameters; $u + (D-u) = D$ follows from that leaf's valid side condition. After (R6), the unrestricted-template expansion supplies such valid restricted leaves, so all offsets are nonnegative.

The existing induction on a derivation now proves that its compiled word either halts or returns the desired formula while preserving the protected old stack and clearing the caches. A refutation gives

$$W_d = \mathsf{AX}(0,0,0)\,C_k(d)\,\mathsf{S},$$

which halts from every incoming command store. The leading AX clears the two caches; it need not clear the carried stack. The alias $p(0,0) = 0$ and the possibility of an earlier halt are handled by the same compiler identities.

There is an essential distinction here: F_k embeds calculus derivations, but it does not conjugate source stores or machine trajectories. It preserves neither ordinary addition nor ordinary Cantor-stack encoding. One must compile a *new* target-radix command word from the translated derivation. In particular, no identity such as $F_k(D-u) = F_k(D) - F_k(u)$ is claimed or needed.

The command-word encoding and the scheduler are unchanged. Every positive continuation pointer strictly decreases during its current candidate, fresh candidates are $2, 3, \ldots$, and an AX-prefixed word has candidate code at least three. Thus PA proves that the universally halting word is eventually reached, or a preceding candidate has already halted. The arbitrary carried command store causes no difficulty.

1.6 Binding the changed arithmetic macro to the machines

The fixed source block called `node61` is the four-increment body in the formula-construction routine `Cons`. Its only direct use is inside the decrementing loop `node62-node63`. Replacing that body by k increments changes the loop to

$$(T, Y) \longmapsto (T + k, Y - 1) \quad (Y > 0),$$

with the zero branch exiting. After $i \le Y_0$ iterations its invariant is

$$T = T_0 + ki, \qquad Y = Y_0 - i,$$

and all unrelated registers are preserved. Induction on Y_0 proves termination with $(T_0 + kY_0, 0)$. Counting the decrement, k increments and return jump gives exactly $(k+2)Y_0 + 1$ primitive source steps, including the final zero test.

The preceding unpair and Cantor-pair macros are unchanged. For the following source-macro contract, the four displayed coordinates are (T, W, X, Y), not the command-store coordinates (t, w, d, h) used above. The scratch registers satisfy $X = Y = 0$ on entry. The new Cons routine sends

$$(t, [a]w, 0, 0) \longmapsto (I_k(a, t), w, 0, 0),$$

preserving every other register. The fixed additional increments for tags one, two and three remain unchanged, so the same free-term emitter calculation produces the heads displayed above, now with the radix-k constructors. All formula-construction uses, including the MP comparison, pass through

this Cons routine. Ordinary stack push and the scheduler's Cantor coding do not pass through the formula-multiplier loop.

The source-binding check makes that separation explicit: there are exactly 61 occurrences of Cons and exactly 61 occurrences of its multiplier body, at the corresponding fixed relative path in each Cons occurrence. There are no other uses of that body. Each Cons has the same 21-site unpair/pair prefix followed by one decrement, k increments and one back-edge jump. All 3429 source sites outside the old increment bodies retain their operations and every successor edge under the checked ordinal relocation. Thus the source has $3673 + 61(k-4)$ sites: 3734 at radix five and 3795 at radix six.

This loop induction is the changed arithmetic-macro proof. The archived file `M6/Source/AXTimes4.lean` states its radix-four instance, with cost $6Y_0 + 1$, and `M6/Source/AXCons.lean` composes it with unpair and pairing. Their fixed multiplier lemma cannot simply be cited as already proving the new macro; the preceding induction supplies the required generalization. The candidate's source expansion and literal primitive contracts must additionally bind the new k-increment body and its successors to the exact table.

For $k = 5, 6$, the supplied finite certificates do this final binding, including the positive duration of every source primitive and the complete blank-tape startup frame. PA formalizes their finite row equalities and the register-loop inductions. The same sound-store annotation used in the archived calibration works with T_k: both formula registers and every canonical stack entry carry PA proofs, successful MP combines them, AX uses the suppliers above, and a halting Shift exposes code one in one of the two registers. The zero-stack alias is handled by its fixed truth annotation.

Under Con(PA) this invariant prevents a command halt. Positive physical simulation can therefore be iterated past any proposed finite halting time, contradicting determinism. Conversely, a PA refutation is transformed by (R13), compiled into W_d, reached by the scheduler, and converted by the physical contracts into a finite blank-tape halt. These operations are primitive recursive and their correctness is provable in PA. For the delivered exact table M_k, $k = 5, 6$, this gives

$$\boxed{\mathrm{PA} \vdash \left(\neg\, \mathrm{Halt}_{\mathrm{blank}}(M_k) \leftrightarrow \mathrm{Con}(\mathrm{PA})\right).} \tag{R14}$$

The statement is an ordinary mathematical PA-provability result with executable finite certificates for the candidate-specific program facts. The new radix endpoints are not claimed to have been exported as newly kernel-checked Lean theorems.